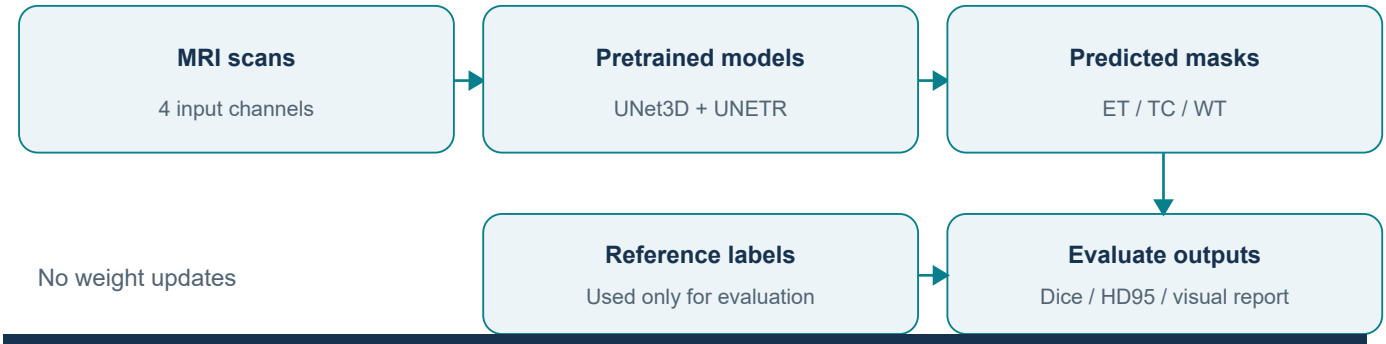
01 / The whole idea

Start here: you do not need a machine-learning background.

Your project: use two already-trained models to mark tumor regions in real brain MRI, compare their masks with reference annotations, and explain the results. This is **segmentation**: locating regions voxel by voxel, not merely classifying a scan as tumor/no tumor.



Term	Plain meaning in this project
AI / machine learning / deep learning	AI is the broad field; machine learning learns patterns from examples; deep learning uses neural networks with many layers.
Model / architecture	A model is a numerical mapping from inputs to outputs. Its architecture is the arrangement of operations and layers.
Weights / parameters	Learned numbers that control the mapping. Here they come from released checkpoint files.
Training / inference	Training changes weights using examples and errors. Inference applies fixed weights to a new input. We ran inference only.
Prediction / reference	The model produces a prediction. The dataset supplies a separate annotated reference used to score it.

What you actually completed

Four patients × two models = eight evaluations on a Kaggle Tesla T4 GPU. The outputs are full-volume masks, Dice and HD95 scores, comparison images, animations and reports. No training or fine-tuning was performed.

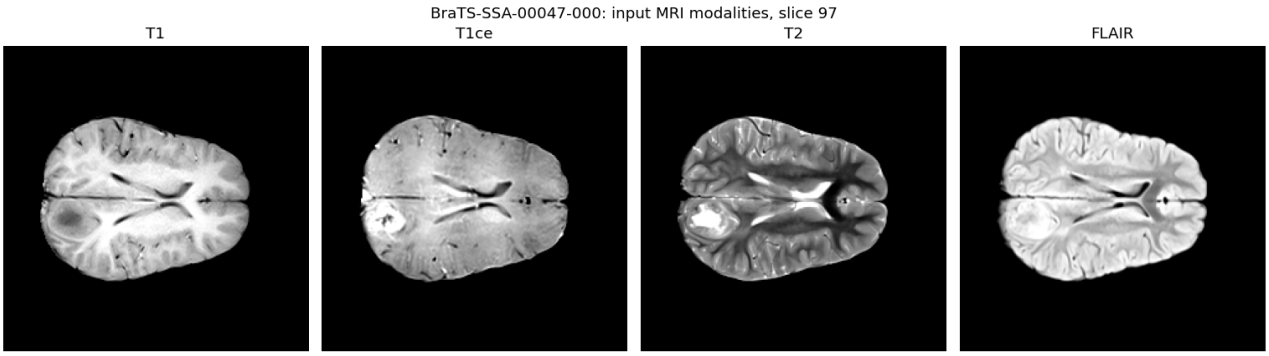
Your contribution and the original authors

Prakhar Gupta (ZekRock18), with teammate Aditya Dubey: Kaggle integration, data adaptation, pretrained evaluation, visualization and verified delivery. Model architectures and pretrained weights are credited to the original Brain_Tumor_Benchmark project and MohanadKombar checkpoints.

Say it back: “I built an end-to-end evaluation and visualization workflow around two released pretrained segmentation models.”

02 / Understand the MRI data

One patient is a set of aligned 3D volumes, not one ordinary photograph.



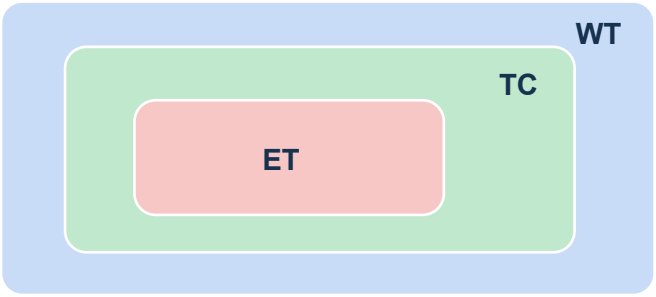
Actual input preview from patient 00047. T1, contrast-enhanced T1 (T1ce), T2 and FLAIR are different MRI contrasts of the same anatomy. They provide complementary intensity patterns.

Pixels, voxels and channels

A pixel is one location in a 2D picture. A voxel is one location in a 3D grid. An axial slice is one plane through the grid. Each sampled scan is 240 × 240 × 155: 8,928,000 voxels. Stacking four modalities gives an array with shape **(4, 240, 240, 155)**. The leading 4 means channels, not four patients.

What is a NIfTI file?

A .nii or compressed .nii.gz file stores an image array plus metadata such as voxel spacing and spatial orientation. The image values are intensities; the segmentation file contains integer labels. The suffixes t1n, t1c, t2w and t2f map to T1, T1ce, T2 and FLAIR. seg is the reference mask.



ET is included in TC.

TC is included in WT.

These masks overlap;

they are not exclusive classes.

Schematic, not a patient image. ET = enhancing tumor; TC = tumor core; WT = whole tumor. The implementation remaps label 3 to 4, then uses ET={4}, TC={1,4}, WT={1,2,4}. Independent confirmation of dataset annotation semantics remains a limitation.

How to read the colors

The overlay paints WT first, TC next and ET last. Red shows ET; the visible green is TC outside ET; visible blue is WT outside TC. However, each complete overlapping mask is evaluated separately. Colors are display choices, not original MRI colors.

Say it back: “Each patient has four MRI input channels plus a separate reference label volume. ET, TC and WT are nested tumor regions.”

03 / Prepare the input and weights

The same architecture and preprocessing must match the checkpoint.

1. Find complete patients

The notebook discovers matching files by patient ID and keeps cases with all four modalities and a reference segmentation. Discovery found 146 complete cases. Seed 42 selected four; selected_cases.csv records exactly which ones. They happened to include two Glioma and two Other Neoplasms cases. This was not a stratified clinical study.

2. Normalize intensities

For each modality, define the brain mask as intensity > 0. Clip intensities to the 0.5th and 99.5th percentiles within that mask. Compute the clipped brain mean and standard deviation, then use **z = (intensity - mean) / (standard deviation + epsilon)**. Set background to zero. This changes intensity scale, not the spatial grid.

Small example: with intensity 140, mean 100 and standard deviation 20, z = 2. This is an arithmetic illustration, not a measured patient value. Epsilon is a tiny number that avoids division by zero.

3. Load the learned model

Construct each architecture using its saved configuration, then load its state dictionary with **strict=True**. The state dictionary maps parameter names to tensors (multidimensional arrays of numbers). Strict loading checks that names and shapes match. A SHA-256 hash identifies the exact checkpoint file. Parameter count or fast execution alone cannot establish checkpoint authenticity.

Operation	Meaning
model.eval()	Switch training-dependent layers, such as dropout, to evaluation behavior. It does not train or load weights.
Disable gradients	Avoid storing the computation graph used for weight updates; reduces inference overhead.
GPU + mixed precision	CUDA runs numerical operations on the GPU; mixed precision uses lower precision for suitable operations. It does not mean multiple GPUs are used.

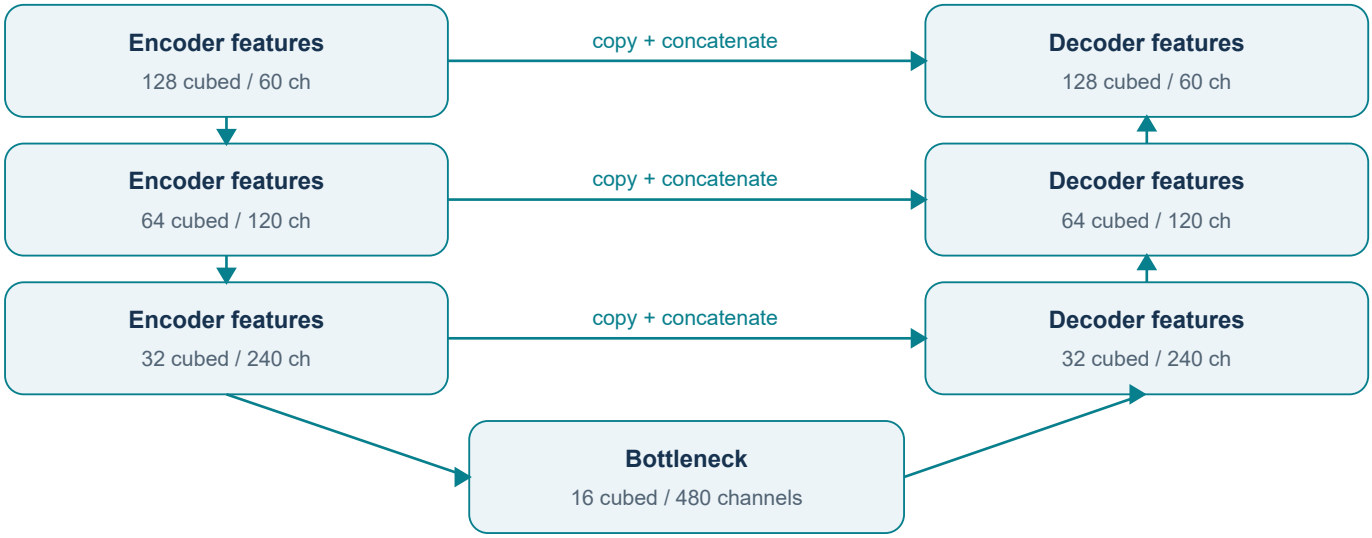
What training would have done

Training repeats: predict, calculate a loss (numerical error against labels), backpropagate gradients (how that error changes with each weight), then use an optimizer to update the weights. A batch is the examples processed together; an epoch is one pass through a training set. Fine-tuning continues learning from existing weights. None of these weight-update steps ran here.

Say it back: “Our checkpoints already contain learned weights. We normalize scans, load compatible weights strictly and run evaluation mode without updates.”

04 / Model one: UNet3D

Convolutions learn local 3D patterns; the decoder restores spatial detail.



Actual checkpoint widths: 60 → 120 → 240 → 480 channels, not the class default widths. Spatial sizes above apply to one 128-cubed inference window. Each arrow across the diagram is a skip connection.

A. Convolution: learn a small pattern detector

A 3 × 3 × 3 filter moves over the volume. At each location it multiplies nearby values by learned weights and sums them, across input channels. Multiple filters produce multiple feature maps, which can represent useful patterns. “Channels” inside the network are learned features, not extra MRI scans.

B. Encoder: smaller grids, richer features

The encoder uses four levels of convolution blocks. Max pooling keeps the largest value in each 2 × 2 × 2 feature neighborhood. Three pooling steps halve the grid: 128 → 64 → 32 → 16. More channels represent richer features at coarser resolutions. Each block uses two convolutions, GroupNorm (4 groups here), LeakyReLU and a residual addition.

C. Decoder: recover where each region belongs

Transposed convolutions upsample 16 → 32 → 64 → 128. The decoder concatenates features from the matching encoder level: these long skip connections restore fine spatial information. More convolution blocks combine the features. A final 1 × 1 × 1 convolution produces three logits per voxel.

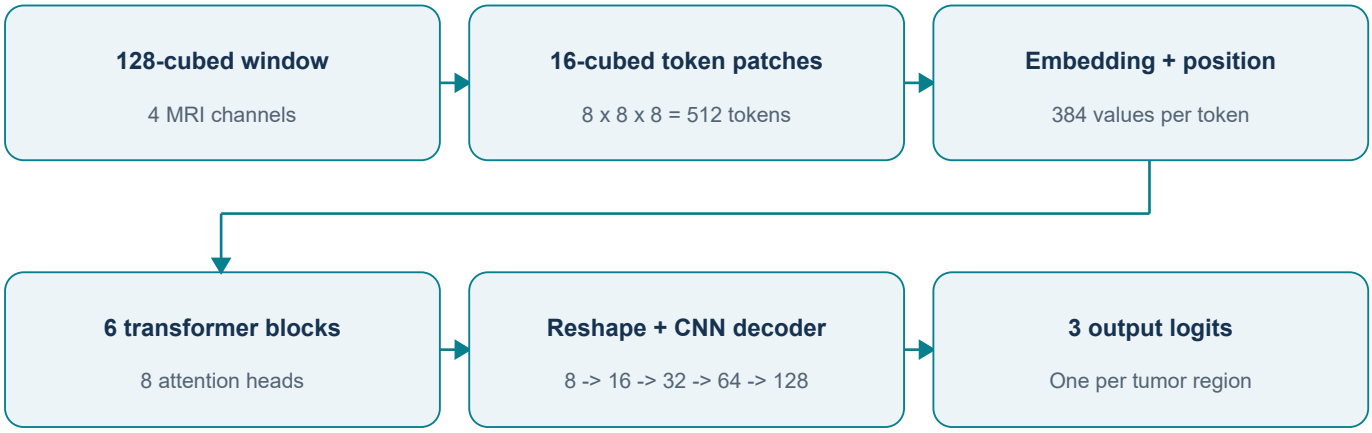
The small terms, decoded

Activation: a nonlinear function that enables complex mappings; LeakyReLU keeps a small slope for negative values. **GroupNorm:** normalizes feature groups within a sample. **Residual addition:** adds a block input to its transformed output. This differs from the U-Net skip, which concatenates encoder and decoder channels.

Say it back: “UNet3D compresses a volume into learned features, then upsamples them. Skip connections preserve location details. Our model has 19,992,603 parameters.”

05 / Model two: UNETR

A transformer shares information among patches; a CNN decoder produces voxel masks.



Verified checkpoint: token patch size 16, embedding dimension 384, six blocks, eight heads. A 128-cubed window contains $(128/16)^3 = 512$ tokens. This is the repository's custom UNETR, not a generic interchangeable checkpoint.

A. Turn image patches into tokens

A strided 3D convolution projects each 16 × 16 × 16 patch into a vector of 384 numbers: its embedding. Learned position embeddings tell the network where each token belongs. Without position information, the token features alone would not explicitly identify their locations.

B. Self-attention: share relevant information

For each token, learned projections create a **query** (what information it seeks), a **key** (what it matches) and a **value** (information it contributes). Query-key dot products produce scores; softmax converts them into weights that sum to one. A weighted sum of values updates the token. This lets tokens interact across the current window, not automatically across the entire patient volume.

Formula: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d})\, V$.
Q, K and V are learned projections of the same token sequence; d is the number of features per head. Here $d = 384/8 = 48$. Eight heads can learn different relationships; they are not eight separate models.

C. Repeat and decode

Each of six blocks uses LayerNorm, attention and residual addition, then another LayerNorm and a feed-forward network (384 → 1536 → 384, with GELU activation), plus residual addition. LayerNorm normalizes features within each token. Layer-3 and final layer-6 features feed the decoder, which reshapes tokens to an 8^3 grid and upsamples to 128^3. Original-input features provide another skip. The CNN decoder uses GroupNorm; the transformer encoder uses LayerNorm.

Say it back: "UNETR represents patches as tokens, relates them using attention and decodes the features back to masks. Our model has 19,327,907 parameters."

06 / From windows to a full mask

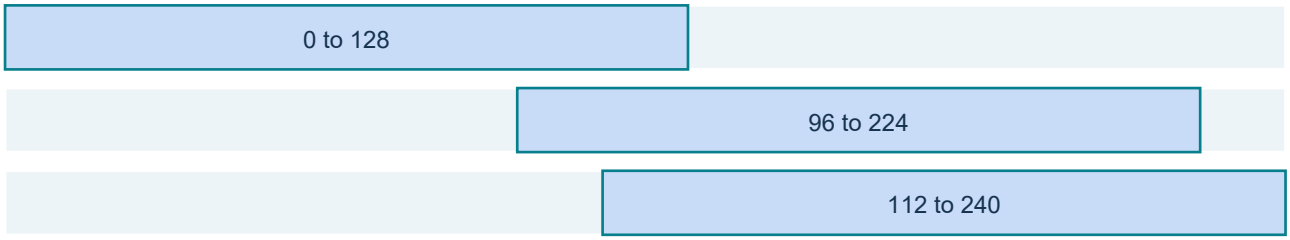
There are two different patch sizes: inference windows and UNETR token patches.

Size	Role
240 × 240 × 155	The full patient volume in this demonstration.
128 × 128 × 128	The inference window fed to either model.
16 × 16 × 16	The smaller token patch used inside UNETR only.

Why not send the whole volume at once?

Large 3D feature maps consume memory. Sliding windows let the model handle manageable pieces while retaining the full patient grid. Neighboring windows overlap by a nominal 25%. The final window is shifted to cover the edge, so its overlap can be larger.

A 240-voxel axis: three 128-voxel windows cover it.



Logit → sigmoid → blended probability → mask

Each model returns three raw scores, called logits, at each voxel. Sigmoid maps a logit z to **1 / (1 + exp(-z))**, between 0 and 1. Each window's sigmoid output is blended into the full grid. A value greater than 0.5 becomes foreground. These outputs are model scores, not guaranteed calibrated medical probabilities.

Gaussian blending prevents hard joins

For a voxel covered by several windows, compute **sum(weight × probability) / sum(weight)**. Gaussian weights favor window interiors. For example, predictions 0.8 and 0.4 with weights 0.75 and 0.25 blend to 0.7, then become foreground. This is an illustration, not an actual stored voxel.

Fair comparison and practical details

Both models use the same selected patients, preprocessing, threshold and evaluation procedure. The notebook uses MONAI when available and a tested built-in fallback otherwise. Masks are thresholded independently; no explicit post-processing enforces ET inside TC inside WT. Inference uses the selected T4 device, not a two-GPU distributed run.

Say it back: "We predict overlapping 128-cubed windows, Gaussian-blend their sigmoid scores, then threshold the full-volume maps."

07 / Know what the metrics mean

Dice asks “how much overlap?”; HD95 asks “how far apart are the boundaries?”

Dice: region overlap, from zero to one

Dice = 2 × shared foreground voxels / (predicted foreground + reference foreground). A score of 1 means exact overlap. If a prediction has 100 voxels, the reference 120, and 90 are shared, Dice = 180/220 = 0.8182. This is not 81.82% classification accuracy and not the chance that the patient has a tumor.

Why not ordinary voxel accuracy?

Most voxels may be background. A model that predicts background everywhere could score highly on accuracy while missing the tumor. Dice focuses on foreground overlap. Still, a good Dice can hide important local errors, so we also inspect images and boundary distance.

HD95: boundary distance in millimetres

Extract both mask surfaces. For every predicted surface point, find its nearest reference-surface distance; also calculate the reverse direction. Pool these distances and take the 95th percentile. This is HD95 in this implementation. Lower is better. Header voxel spacing converts grid distances into physical units. It is not the mean of the worst 5% of errors.

Why we report both

Two large masks may overlap strongly yet have a small, distant mismatch. In actual case 00170, UNet3D ET Dice is 0.9668 but ET HD95 is 13.63 mm. High overlap does not guarantee that all boundaries are close. Conversely, percentile-based HD95 can ignore a small fraction of extreme errors.

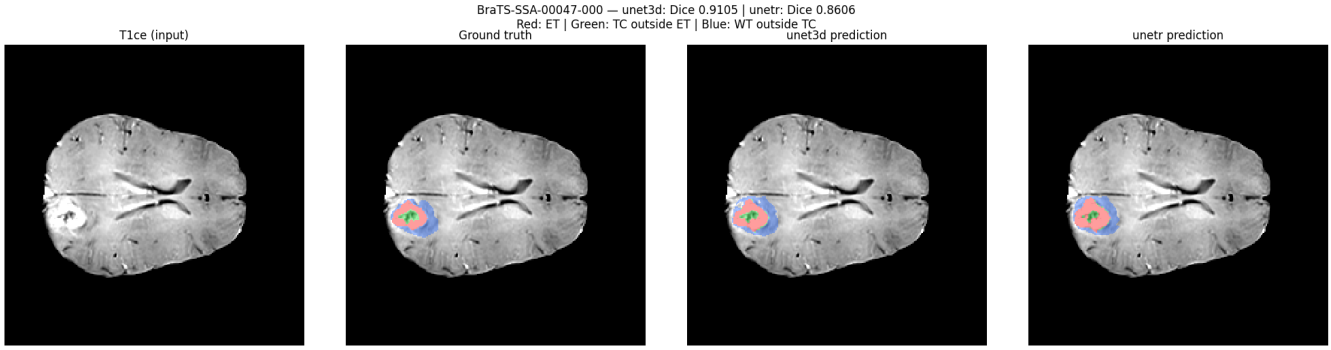
Edge case / averaging	Implemented behavior
Both masks empty	Dice = 1; HD95 = 0.
Only one mask empty	Dice is zero or near zero with numerical smoothing; HD95 uses the repository’s fixed nominal-volume diagonal penalty, about 373.13 mm.
Mean Dice / HD95	Average the three region scores per case, then average across four cases. Every case has equal weight; voxels are not pooled into one global Dice.

Say it back: “Dice rewards matching regions; HD95 penalizes separated boundaries. I need both metrics and the visual overlays to assess a case.”

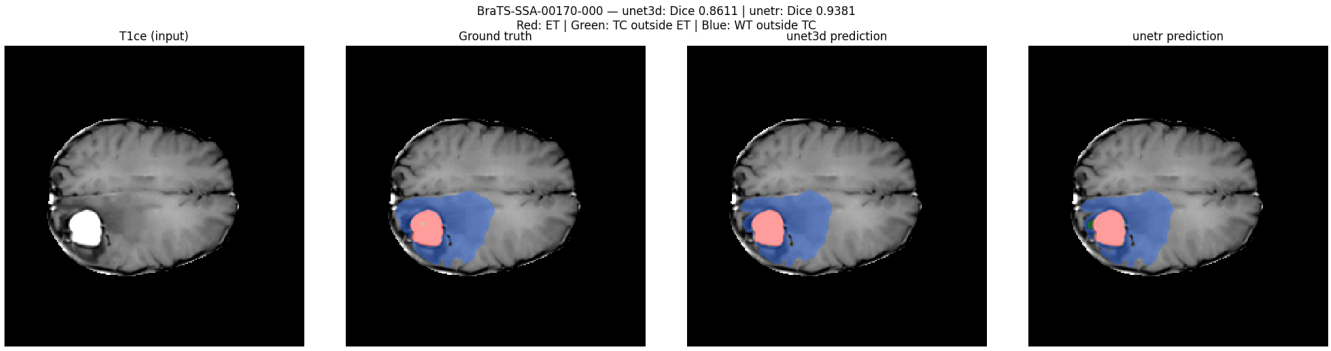
08 / Read your actual results

These measurements come from the completed Kaggle run, not a synthetic test.

Model	ET Dice	TC Dice	WT Dice	Mean Dice	HD95 mm
UNet3D	0.8611	0.7868	0.9158	0.8546	6.3594
UNETR	0.8618	0.8281	0.8788	0.8562	5.6408



Case 00047: MRI → reference → UNet3D → UNETR. Mean Dice: 0.9105 vs 0.8606. The displayed axial slice has the largest reference WT area; scores cover the entire volume.



Case 00170: mean Dice 0.8611 vs 0.9381; mean HD95 14.18 vs 6.75 mm. Model performance changes with the patient. Open the saved GIFs to inspect more than one slice.

What you can say

Both models produced segmentations for all four cases. UNETR has slightly higher mean Dice and lower mean HD95 in this sample; UNet3D has higher mean WT Dice. Each model has the higher patient-mean Dice on two cases. The overall Dice gap is only about 0.0017, not evidence of a generally superior model.

What the sample cannot prove

Four cases are not clinical validation. Training-set overlap was not independently excluded, so do not promise unseen patients or a proven out-of-distribution benchmark. Label semantics need independent confirmation. The original author's validation scores, 0.8694 and 0.8362, are different results. No training gains or controlled speed/peak-memory claims are made here.

Say it back: "Our measured means are 0.8546 and 0.8562 Dice. The result demonstrates a working pretrained evaluation, not a universal winner."

09 / Connect the explanation to code

Use this page to narrate the notebook from start to finish.

Stage	What happens and where to look
1. Setup	Imports libraries, fixes seed 42, checks CUDA. GPU guard stops an unintended CPU run. PyTorch runs models; NumPy handles arrays; nibabel reads NIfTI.
2. Model definitions	The notebook contains UNet3D and UNETR classes from the original project. Source: reproducibility/repo/models/.
3. Preprocess + metrics	Normalization, region conversion, Dice and HD95. Original source: data/preprocessing.py and training/metrics.py in the source snapshot.
4. Load weights	Read checkpoint configuration, construct matching architectures, load strictly and record hashes. A .bin suffix is a saved file, not a model architecture.
5. Discover / select	Find all modalities and labels, choose cases and record their identities. Inputs are attached on Kaggle, not downloaded to your laptop.
6. Predict / score	Run both models over the full volume through sliding windows; compute ET/TC/WT metrics against the labels.
7. Export / verify	Save CSVs, images, animated slices and report.html. SUCCESS.json lists expected artifacts; END-TO-END COMPLETE records successful completion.

The four files to understand first

results/executed.ipynb: historical execution and outputs, all 15 code cells completed.
results/brats_africa_results.csv: one row per patient and model.
results/run_metadata.json: settings, GPU and checkpoint identities.
results/report.html: the offline visual demonstration.

How to rerun or present

Use the public Kaggle notebook, attach BraTS-Africa and the checkpoint input, select a GPU and run all cells. Confirm actual CUDA output and final artifacts rather than assuming a GPU request succeeded. For a presentation, open the saved HTML report and PDFs; no live GPU is needed to view existing results. A new run is not implied by viewing the report.

Say it back: “The executed notebook proves what ran; the CSV gives exact numbers; metadata explains the settings; the visual report makes the findings understandable.”

10 / Explain it without memorizing

Practice these answers, then close the PDF and explain them in your own words.

What is the project?
A reproducible Kaggle demonstration comparing two released pretrained 3D tumor-segmentation models on four real labeled MRI cases.

Why four inputs and three outputs?
The four inputs are MRI modalities. The three outputs are ET, TC and WT region scores at every voxel; they are not three patient diagnoses.

What is the main model difference?
UNet3D extracts local volumetric features with convolutions. UNETR relates patch tokens with attention, then uses a CNN decoder to produce dense masks.

Did you train either model?
No. The original author supplied trained checkpoints. We integrated loading, data handling, inference, evaluation, visualization and verification.

Why strict loading and hashes?
Strict loading checks structural compatibility. SHA-256 identifies the exact file. Neither runtime nor parameter count alone proves checkpoint identity.

Why 128 and 16?
128 cubed is the window fed to both models. 16 cubed is UNETR's internal token patch; each window therefore contains 512 tokens.

Does a higher Dice mean clinically better?
Not by itself. Boundary errors, sample size, label validity, population coverage and clinical validation all matter. This is an educational demonstration.

What would you improve next?
Confirm label semantics, audit training overlap, evaluate a larger predefined sample, quantify uncertainty and measure runtime under controlled conditions.

Your 30-second closing explanation

"I use four MRI modalities as input, normalize them, load the exact pretrained UNet3D and UNETR weights, and predict overlapping 3D windows. After blending and thresholding, I compare ET, TC and WT against the supplied labels using Dice and HD95. My four-case run completed on a T4 GPU, with mean Dice 0.8546 and 0.8562. The project includes traceable results and visual reports, while acknowledging the small sample and remaining validation limits."

Project: https://github.com/ZekRock18/brain-tumor-segmentation
Kaggle: https://www.kaggle.com/code/zekrock/brain-tumor-seg-brats-africa

Evidence: saved run metadata, per-case CSV and exact checkpoint configurations; architecture details checked against reproducibility/repo/models/unet3d.py and unetr.py. Original models: MohanadMahran/Brain_Tumor_Benchmark; pretrained weights: MohanadKombar on Hugging Face. The other project PDFs provide full source links and reproducibility details.